

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Modelling Comparative Analysis
Assignment

COURSE NAME:	ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS	COURSE CODE:	MLA0101
---------------------	---	-------------------------	----------------

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.* (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A
Modelling Comparative Analysis Assignment

Code of Conduct:

*I **Yoga Madha A-192425058** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

1. Banking Loan Eligibility – Knowledge Representation and Prolog

Problem Statement & Objectives

A bank needs an intelligent system to determine whether a customer is eligible for a loan. The decision is based on factors such as income, credit score, employment status, existing loans, and repayment history.

The main objective is to represent this decision-making problem using different AI knowledge representation techniques and compare their reasoning capabilities.

Objectives:

- Identify important banking facts and conditions.
- Represent loan decisions using production rules.
- Model the problem using propositional logic.
- Represent customers and their relationships using first-order logic.
- Implement the knowledge base using Prolog.
- Demonstrate forward and backward chaining.
- Compare the different approaches.
- Identify the most suitable model for loan eligibility.

Domain Knowledge & Facts

Entities

- Customer
- Loan
- Bank
- Employment
- Credit Record
- Repayment History

Important Attributes

- Customer income
- Credit score
- Employment status
- Existing loans
- Repayment history
- Loan amount
- Loan eligibility

Example Facts

A customer may have:

Income = High

Credit Score = Good

Employment = Stable

Existing Loan = No

Repayment History = Good

Based on these facts, the system can conclude:

Loan Eligible

Possible Conclusions

- Loan Eligible
- Loan Not Eligible
- Requires Further Verification
- High Risk Customer

Production Rule Model

Production rules use **IF–THEN** statements.

Rules

IF income is high AND credit score is good

THEN customer is eligible.

IF employment is stable AND repayment history is good

THEN customer is eligible.

IF credit score is poor

THEN customer is not eligible.

IF repayment history is poor

THEN customer is high risk.

IF existing loans are high AND income is low

THEN customer is not eligible.

Example

Income = High

Credit Score = Good

Employment = Stable

Repayment History = Good

The rules are applied and the system derives:

Customer → Loan Eligible

Propositional Logic Model

Propositional logic represents facts using propositions.

Let:

I = Customer has sufficient income

C = Customer has good credit score

E = Customer has stable employment

R = Customer has good repayment history

L = Customer is loan eligible

P = Customer has poor credit score

Logical Rules

$I \text{ AND } C \rightarrow L$

$E \text{ AND } R \rightarrow L$

$P \rightarrow \text{NOT } L$

Example

If:

$I = \text{True}$

$C = \text{True}$

Then:

$I \text{ AND } C = \text{True}$

Therefore:

$L = \text{True}$

So the customer is **loan eligible**.

First-Order Logic Model

First-order logic provides a more expressive representation using **predicates, variables, and relationships**.

Predicates

Income(Customer, Level)

CreditScore(Customer, Score)

Employment(Customer, Status)

Repayment(Customer, History)

ExistingLoan(Customer, Amount)

Eligible(Customer)

Rules

$\forall x$:

Income(x, High) AND CreditScore(x, Good)

$\rightarrow \text{Eligible}(x)$

$\forall x$:

Employment(x, Stable) AND Repayment(x, Good)

$\rightarrow \text{Eligible}(x)$

$\forall x$:

CreditScore(x, Poor)

$\rightarrow \text{NOT Eligible}(x)$

Example

Income(Ravi, High)

CreditScore(Ravi, Good)

Employment(Ravi, Stable)

Repayment(Ravi, Good)

The system can derive:

Eligible(Ravi)

Prolog Model & Implementation

Prolog is used to create facts and rules and automatically perform logical reasoning.

Prolog Facts

income(ravi, high).
credit_score(ravi, good).
employment(ravi, stable).
repayment_history(ravi, good).
existing_loan(ravi, no).

Prolog Rules

eligible(Customer) :-
 income(Customer, high),
 credit_score(Customer, good).

eligible(Customer) :-
 employment(Customer, stable),
 repayment_history(Customer, good).

not_eligible(Customer) :-
 credit_score(Customer, poor).

not_eligible(Customer) :-
 repayment_history(Customer, poor).

Query

?- eligible(ravi).

Output

true.

Therefore:

Ravi is eligible for the loan.

Another query:

?- not_eligible(ravi).

Output:

false.

Forward & Backward Chaining

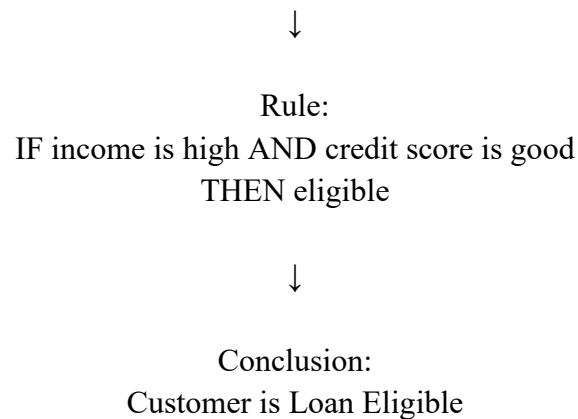
Forward Chaining

Forward chaining starts with **known facts** and applies rules to reach a conclusion.

Income = High
+
Credit Score = Good
↓
Eligibility Rule
↓
Loan Eligible

Inference Sequence

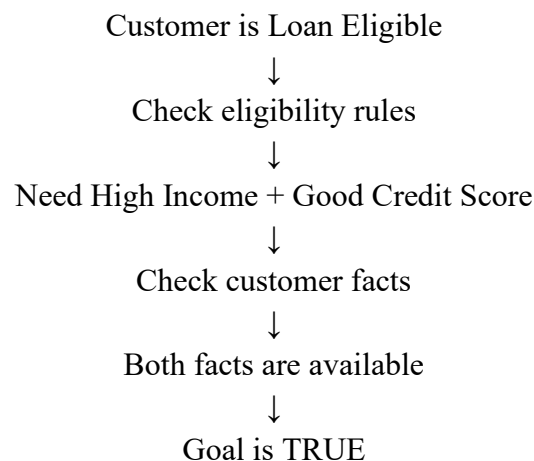
Fact 1: Income is High
Fact 2: Credit Score is Good



Backward Chaining

Backward chaining starts with a **goal** and works backward to find supporting facts.

Goal:



Prolog mainly uses **backward chaining** for queries.

Results, Discussion & Recommended Model

The different knowledge representation approaches can successfully represent the banking loan eligibility problem.

Production rules are easy to understand and are useful for representing banking policies.

Propositional logic is simple but cannot effectively represent different customers and complex relationships.

First-order logic provides better representation of customers, properties, and relationships, but its implementation is more complex.

Prolog combines facts, rules, variables, and logical reasoning. It can easily process customer information and answer eligibility queries.

Recommended Model

Prolog is recommended for the banking loan eligibility system because:

- It is easy to implement.
- It supports facts and rules.
- It provides automatic logical inference.
- It supports backward chaining.
- It is suitable for expert systems.

- It provides understandable reasoning.
- New customers and rules can easily be added.

Limitations

- The system depends on predefined rules.
- Real banking decisions involve many additional factors.
- It does not automatically learn from historical loan data.
- Uncertain or incomplete information is difficult to handle.

Possible Improvements

The system can be improved by adding:

- More financial conditions.
- Risk scoring.
- Loan amount calculation.
- Interest-rate recommendations.
- Historical loan data.
- Machine-learning-based risk prediction.
- A web-based user interface.

Conclusion

The Banking Loan Eligibility problem can be effectively represented using production rules, propositional logic, first-order logic, and Prolog.

Production rules provide simple decision-making, propositional logic handles basic logical conditions, and first-order logic provides a more expressive representation of entities and relationships.

Prolog provides an effective combination of **knowledge representation and automated reasoning**. Therefore, Prolog is the most suitable approach for developing the proposed loan eligibility expert system.

References

- Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*.
- Ivan Bratko, *Prolog Programming for Artificial Intelligence*.
- George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*.
- SWI-Prolog documentation.

GitHub

<https://github.com/YOGAMADHA/MLA0101-Artificial-Intelligence>

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF–THEN rules	True/False propositions	Predicates, variables and quantifiers	Facts, rules and predicates
Expressiveness	Medium	Low	Very High	High
Rule Representation	Easy IF–THEN rules	Logical formulas	Predicate-based rules	Rules using Prolog syntax
Inference Mechanism	Rule-based inference	Logical inference	Predicate inference	Automated logical inference
Forward Chaining	Well supported	Possible	Possible	Not the main approach
Backward Chaining	Possible	Limited	Possible	Strongly supported
Handling Relationships	Limited	Poor	Excellent	Excellent
Scalability	Medium	Low	High	High
Explainability	Very High	High	High	High
Ease of Implementation	Easy	Easy	Moderate/Hard	Easy
Suitability for the Problem	Good	Basic	Very Good	Excellent

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.

2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.
6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.

7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				